

ROI-Based Object Detection & Segmentation Pipeline

A high-performance real-time computer vision system that performs **Object Detection, Segmentation, and ROI (Region of Interest) Filtering** using **YOLOv7, OpenCV, RabbitMQ, and Redis**. The system is designed with a distributed architecture that enables parallel processing, efficient frame management, and scalable deployment.

Project Overview

This project processes live video streams by capturing frames, storing them temporarily in **Redis**, and distributing processing tasks through **RabbitMQ**. Separate workers perform object detection and segmentation independently. Detection results are then filtered based on a predefined **Region of Interest (ROI)** before valid detections are saved and displayed.

This architecture minimizes communication overhead, improves processing speed, and supports real-time monitoring applications.

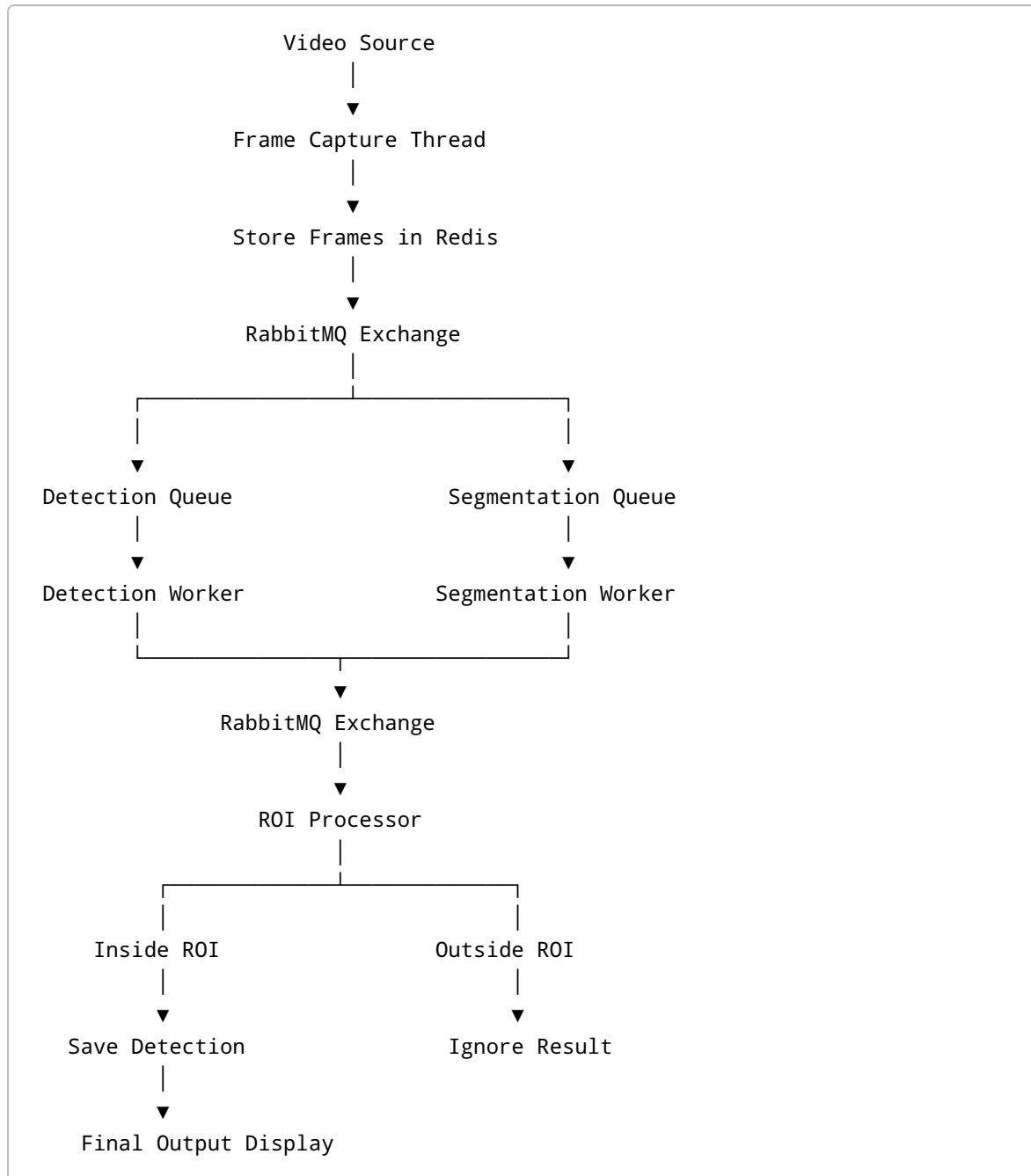
Features

-  Real-time video frame processing
 -  ROI-based object filtering
 -  YOLOv7 Object Detection
 -  Image Segmentation
 -  Parallel processing using multithreading
 -  RabbitMQ message queue integration
 -  Redis in-memory frame storage
 -  Object counting and confidence scores
 -  Modular and scalable pipeline
-

Technologies Used

- Python
 - OpenCV
 - YOLOv7
 - PyTorch
 - Redis
 - RabbitMQ
 - NumPy
 - Threading / Multiprocessing
-

System Architecture



Workflow

1. Frame Capture

- A dedicated thread continuously captures frames from a live camera or video source.
- Each frame is assigned a unique Frame ID.

2. Frame Storage in Redis

- Captured frames are stored in Redis lists (for example, `vehicle_frames` and `safety_frames`).
- Redis acts as a high-speed shared memory accessible by multiple workers.

3. RabbitMQ Message Exchange

- Instead of sending full image frames, only the **Frame ID** and metadata are published to RabbitMQ.
- This reduces network traffic and improves processing efficiency.

Example message:

```
{
  "frame_id": 102,
  "camera_id": 1,
  "timestamp": "2026-06-26 12:10:32"
}
```

4. Detection Worker

The Detection Worker:

- Receives the Frame ID from RabbitMQ
- Retrieves the corresponding frame from Redis
- Runs the YOLOv7 detection model
- Generates object labels, bounding boxes, and confidence scores
- Publishes detection results back to RabbitMQ

Supported object classes include:

- Person
- Car
- Bike
- Helmet
- Safety Jacket
- Fire
- Smoke

5. Segmentation Worker

The Segmentation Worker:

- Retrieves the frame from Redis
- Runs the segmentation model
- Generates pixel-level masks
- Publishes segmentation results for downstream processing

6. ROI Processing

The ROI Processor receives detection results from RabbitMQ and checks whether detected objects lie inside the predefined Region of Interest.

-  Objects inside the ROI are accepted and saved.
-  Objects outside the ROI are ignored.

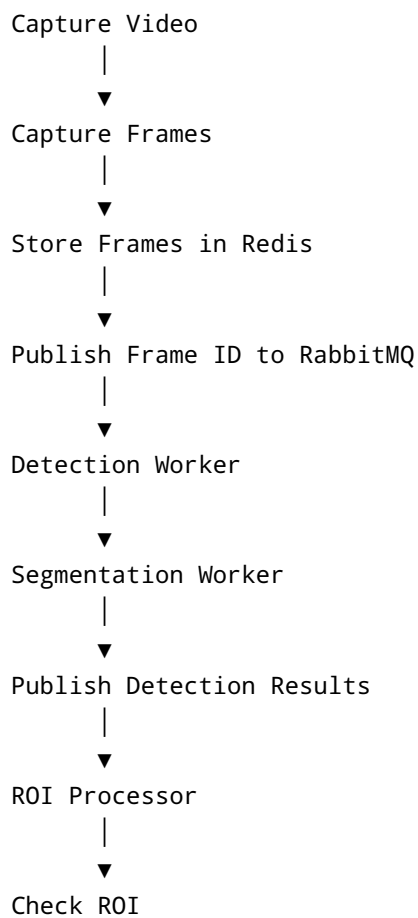
This ensures that only relevant detections are retained.

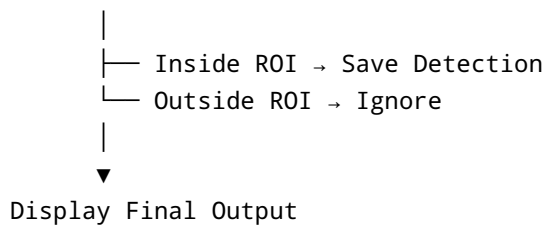
7. Output

The application displays:

- Live video stream
- ROI boundary
- Bounding boxes
- Object labels
- Confidence scores
- Object count

Processing Pipeline





Multithreading

The application uses separate threads for improved performance:

- Frame Capture Thread
- Redis Storage Thread
- Detection Worker Thread
- Segmentation Worker Thread
- ROI Processing Thread
- Display Thread

Benefits

- Parallel execution
- Faster frame processing
- Better CPU utilization
- Improved scalability
- Reduced latency

Project Structure

```
roi-object-detection/
├── models/
│   ├── yolov7.pt
│   └── segmentation_model.pt
├── processors/
│   ├── detection_processor.py
│   ├── segmentation_processor.py
│   └── roi_processor.py
├── redis/
├── rabbitmq/
├── utils/
└── main.py
```

```
| requirements.txt
| README.md
```

Advantages

- High-speed object detection
 - Efficient Redis-based frame sharing
 - Asynchronous communication with RabbitMQ
 - ROI-based filtering reduces false detections
 - Modular and scalable architecture
 - Suitable for real-time industrial applications
-

Applications

- Industrial Safety Monitoring
 - Traffic Surveillance
 - Smart Parking Systems
 - Security Surveillance
 - Warehouse Automation
 - Restricted Area Monitoring
 - Fire and Smoke Detection
-

Future Enhancements

- Multi-camera support
 - Dynamic ROI selection
 - Object Tracking (ByteTrack / DeepSORT)
 - Analytics Dashboard
 - Cloud Deployment
 - Event Notifications
 - Database Integration
 - REST API Support
-

Acknowledgements

This project demonstrates the integration of modern computer vision techniques with distributed messaging systems. By combining **YOLOv7**, **Redis**, **RabbitMQ**, and **OpenCV**, it provides a scalable and efficient solution for real-time object detection, segmentation, and ROI-based monitoring.